Unsupervised Machine Learning

Python Programming for Business Intelligence

Sippo Rossi
May 6th 2026

Learning goals

Theoretical:

- Understanding what is and when to use unsupervised learning
- Knowing what is clustering and dimensionality reduction
- Knowing what are the most common unsupervised machine learning algorithms

Practical:

- Be able to implement and apply unsupervised machine learning to cluster similar data

Motivation

Unsupervised machine learning provides us with powerful tools for a variety of tasks

Unsupervised machine learning can help us

- Help us classify and group data without needing labeled training data
- Reduce the dimensionality of existing data to make it easier and computationally lighter to work with
- Help us detect outliers
- … and more!

Agenda

- Introduction to unsupervised machine learning
- K-Means clustering
- Density-based clustering
- Hierarchical clustering
- Dimensionality reduction
- Conclusion

Unsupervised machine learning

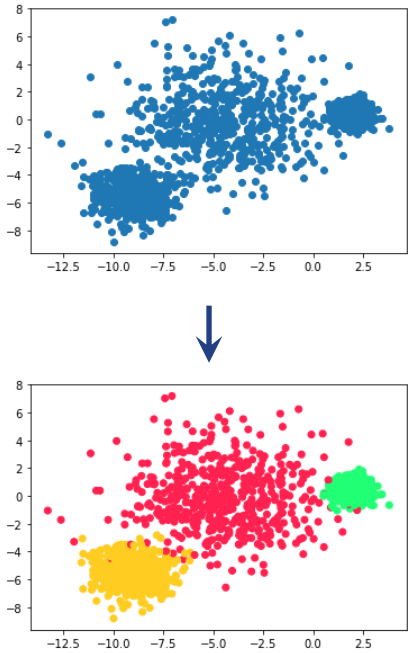
Unsupervised learning

In contrast to supervised ML, in unsupervised ML labels are not provided, and the algorithm learns how to group the data without being explicitly told what the groups are

E.g., grouping a webstore's customers into segments or finding an outlier in transactions, or reducing a high dimension dataset to a lower dimension making it easier to work with

Some common algorithms:

- Clustering: k-means, hierarchical
- Dimensionality reduction: principal components analysis (PCA), t-SNE



Customer segmentation with unsupervised ML

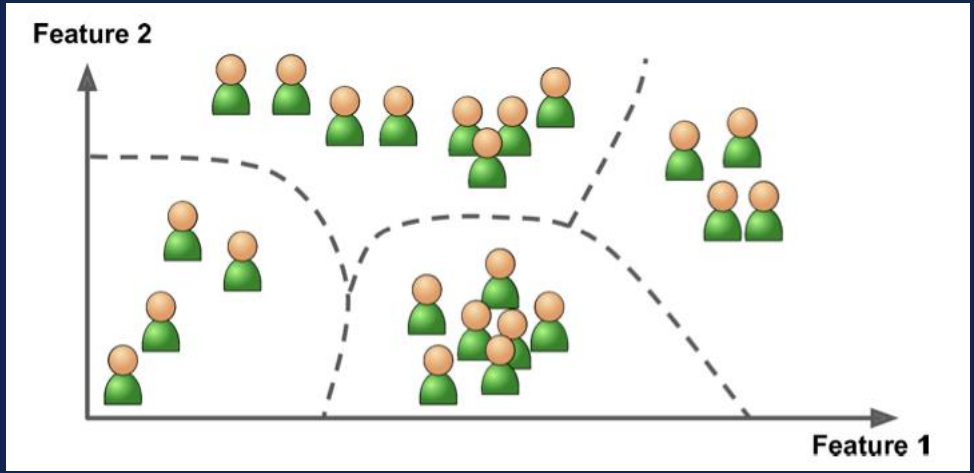


Image from Géron, A. (2019)

Clustering

Input: set of objects described by features x_i.

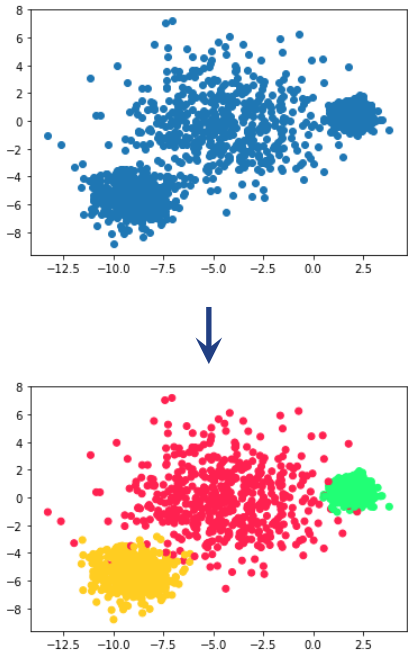
Output: an assignment of objects to “groups”.

Unlike classification, we are not given the “groups”

- The clustering algorithm must discover the groups

Examples of groups we might discover when clustering the webstore’s customers:

- Wealthy customers
- Frugal customers
- Pregnant customers
- Discount hunting customers



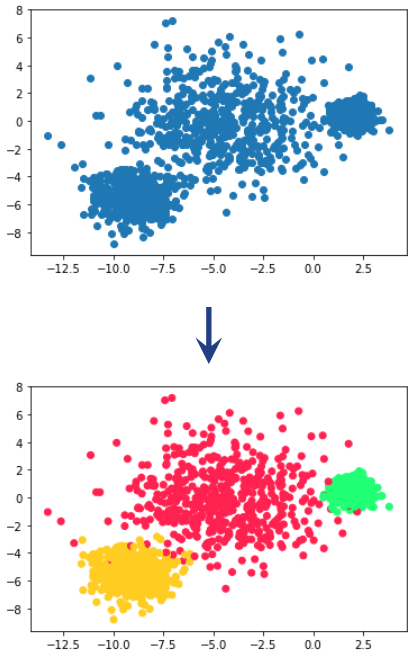
Applications of clustering

Defining market segments by clustering customers

Studying social networks by recognizing communities

Recommendation systems

- Product recommendation (e.g. Amazon)
- Content recommendations (e.g. Netflix)



Other applications of unsupervised ML

Dimensionality reduction:

- As a preprocessing step to reduce number of dimensions while maintaining meaningful properties of the original data

These are beyond the scope of this course, but it is good to know of their existence:

Outlier detection:

- Identify instances with low affinity to all clusters

Semi-supervised learning:

- When few labels are available, perform clustering and propagate the labels to all the instances in the same cluster

K-Means clustering

K-Means clustering

Easy to use and understand

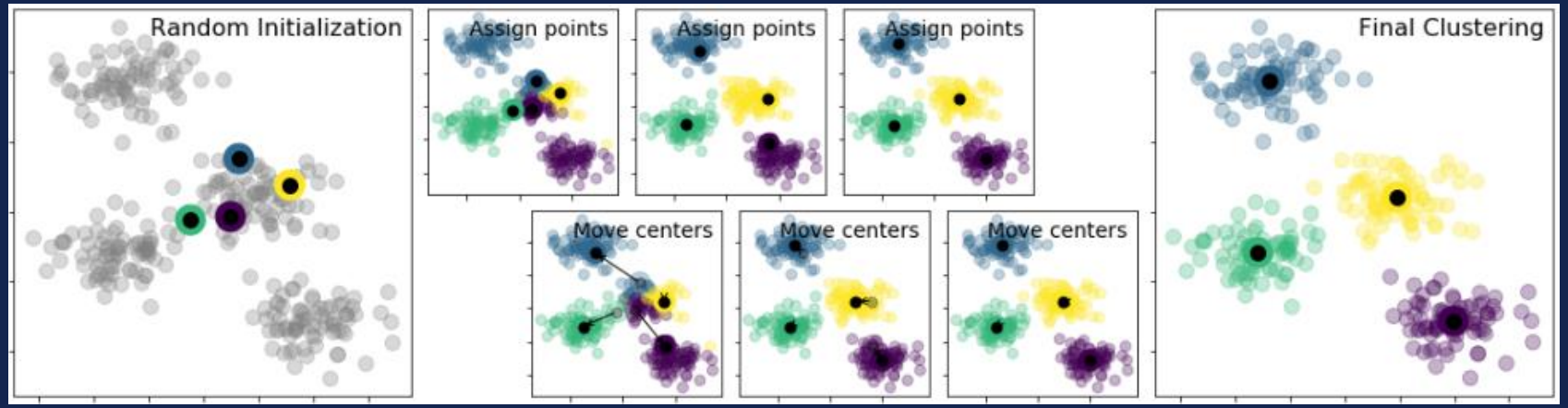
Assigns data points to the nearest mean (cluster center)

What the algorithm does:

1. Initial guess of the center (the “mean”) of each cluster
2. Assign each x_i to its closest mean
3. Update the means based on the cluster assignments
4. Repeat steps 2-3 until convergence

`sklearn.cluster.Kmeans`

K-Means clustering



The math behind K-Means clustering

We can interpret K-Means steps as trying to minimize an objective:

Given a set of observations ($\mathbf{x}_1$, $\mathbf{x}_2$, ..., $\mathbf{x}_n$), the algorithms goal is to partition the *n* observations into *k* sets $\mathbf{S} = \{S_1, S_2, ..., S_k\}$ so as to minimize the within-cluster sum of squares:

$$\underset{\mathbf{S}}{\arg\min} \sum_{i=1}^{k} \sum_{\mathbf{x} \in S_i} \|\mathbf{x} - \boldsymbol{\mu}_i\|^2$$

where $\boldsymbol{\mu}_i$ is the mean of points in S_i

and x is a vector representing the nth observation

Assumptions of K-Means clustering

The center of each cluster is the mean of all samples belonging to that cluster

Each sample is closer to the center of its own cluster than to centers of other clusters

Assumes you know the number of clusters ‘k’.

Issues with K-Means clustering

Final cluster assignment depends on initialization of centers

- Cluster assignments may vary on different runs
- May not achieve global optimum

Each object is assigned to one (and only one) cluster:

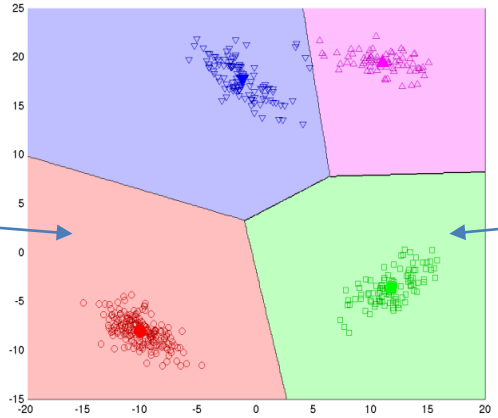
- No possibility for overlapping clusters or leaving objects unassigned
- Fuzzy clustering / soft k-means allows assigning to many

Sensitive to scale

The shape of K-Means clusters

K-means partitions the space based on the “closest mean” into convex regions

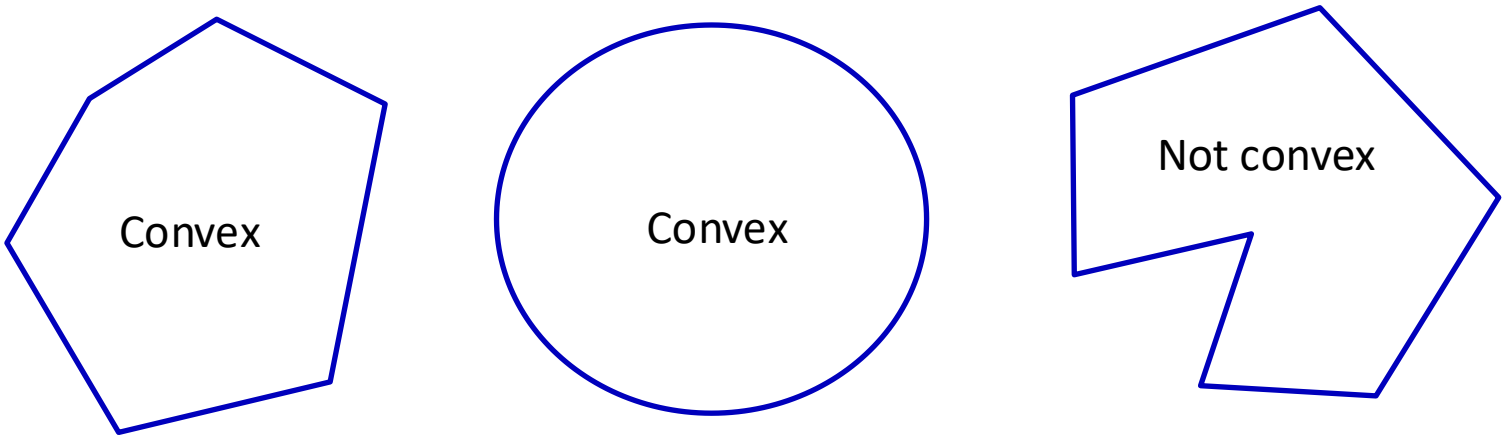
Points here are closest to the red mean



Points here are closest to the green mean

Convex sets

A set is convex if line between two points in the set stays in the set

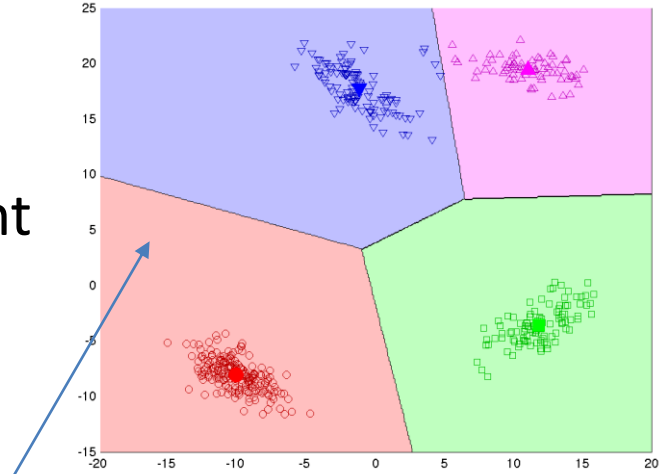


The shape of K-Means clusters

Issues with the shape of K-Means clusters:

1. Real clusters in the data might not be convex

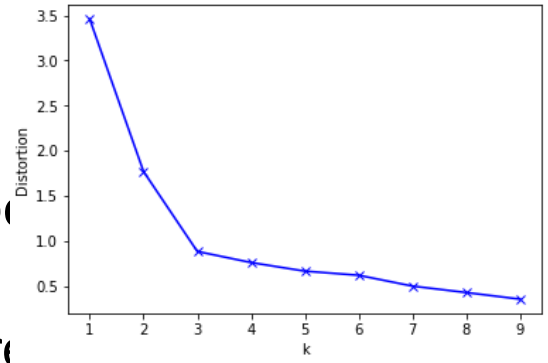
2. Does a data point here really belong in this cluster?



Determining how many clusters with K-Means

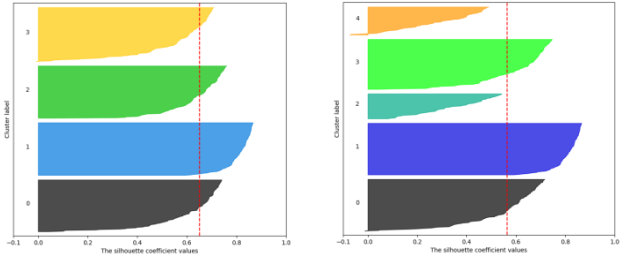
Elbow method:

- Run k-means for several *k*
- Distortion: sum of distances of each p the closest cluster
- Look for *k* where the curve stops decre



Silhouette analysis:

- Explained in Python demo

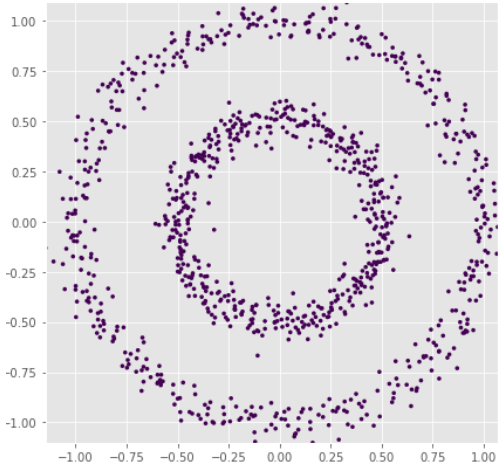


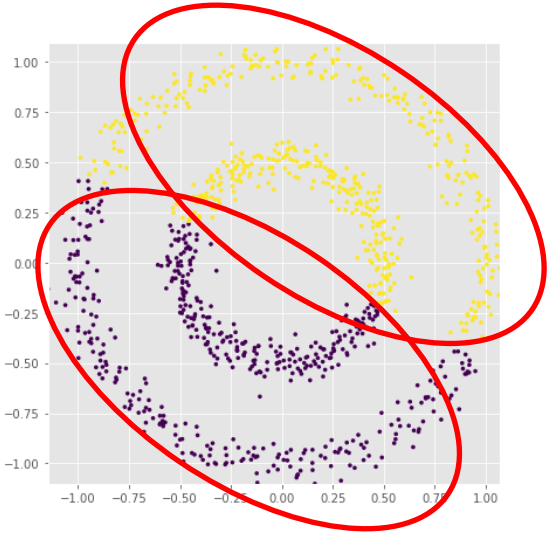
Images from https://pythonprogramminglanguage.com/kmeans-elbow-method/ and https://scikit-learn.org/stable/auto_examples/cluster/plot_kmeans_silhouette_analysis.html

Density-based clustering

Density-based clustering

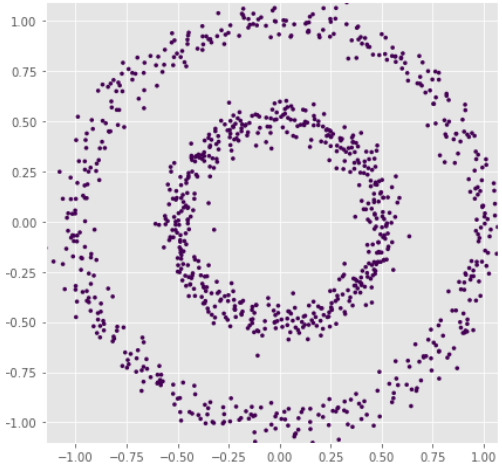
Start by thinking what would happen if you used K-Means clustering on this dataset

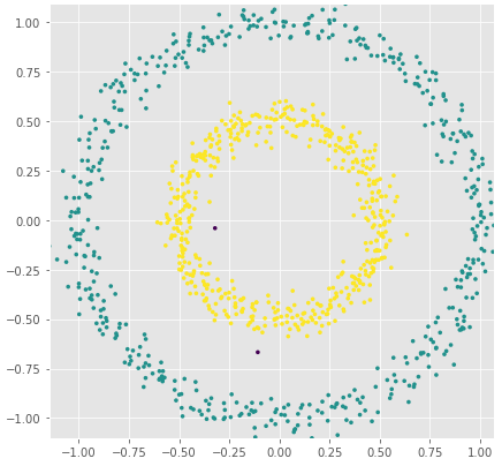




Density-based clustering

Clearly K-Means does not do a good job in clustering this data, but DBSCAN can





DBSCAN

The DBSCAN algorithm clusters continuous regions of high-density data

What the algorithm does:

1. For each instance calculate how many other instances are within distance ε (epsilon) from it
2. If an instance has at least min_samples instances in its ε-neighborhood then it is a *core instance*.
3. Instances in the neighborhood of a core instance belong to the same cluster. (A long sequence of neighboring core instances forms a single cluster.)
4. Any instance that is not a core instance and does not have one in its neighborhood is considered an anomaly

DBSCAN

DBSCAN has two hyperparameters:

Epsilon (ε): distance we use to decide if another point is a "neighbor"

MinNeighbors: number of neighbors needed to say a region is "dense"

`sklearn.cluster.DBSCAN`

How DBSCAN differs from K-Means

Deterministic: always gives the same clusters

No fixed number of clusters ‘k’, determines them by itself

Objects in non-dense regions don’t get clustered

Not trying to “partition” the space

Clusters can be non-convex. I.e. you can find clusters of any shape

DBSCAN vs K-Means

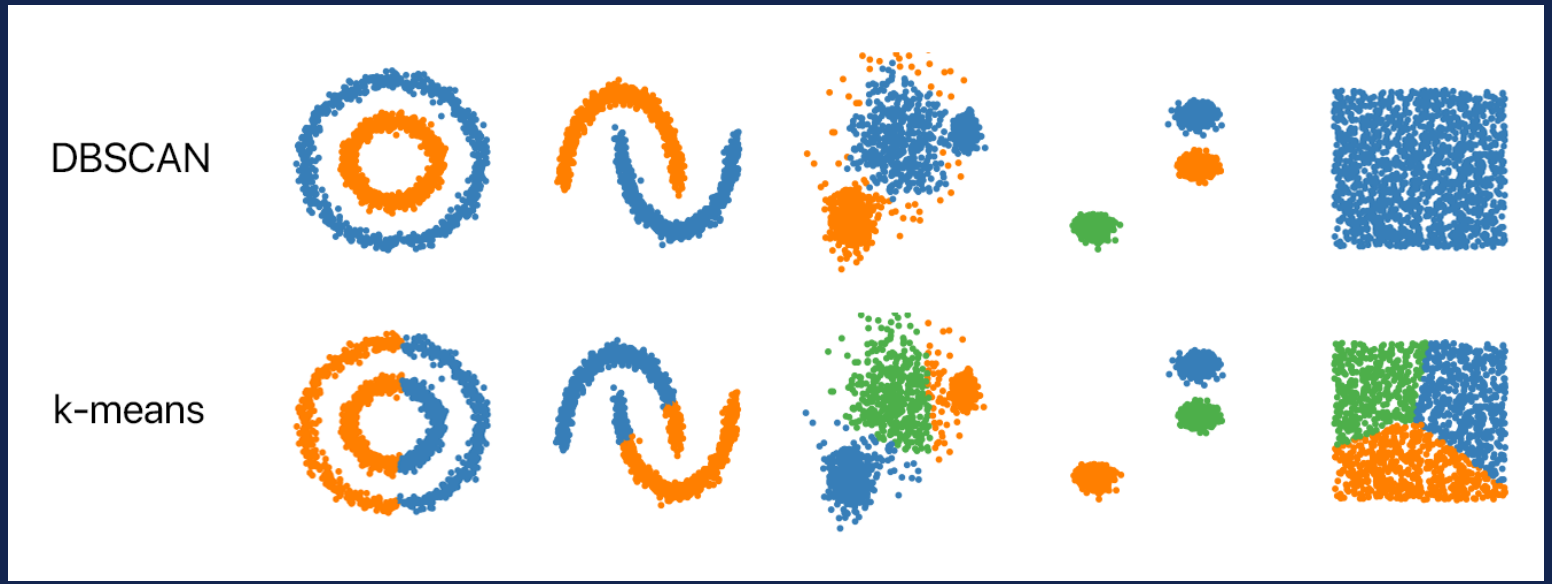


Image from here

Clustering example

Going through *L7_1_kmeans_dbscan.ipynb*

Hierarchical clustering

Divisive Clustering: top-down hierarchical clustering where everything starts in one cluster and are then divided into smaller and smaller clusters

Agglomerative Clustering: hierarchical clustering using a bottom-up approach where each observation starts in its own cluster

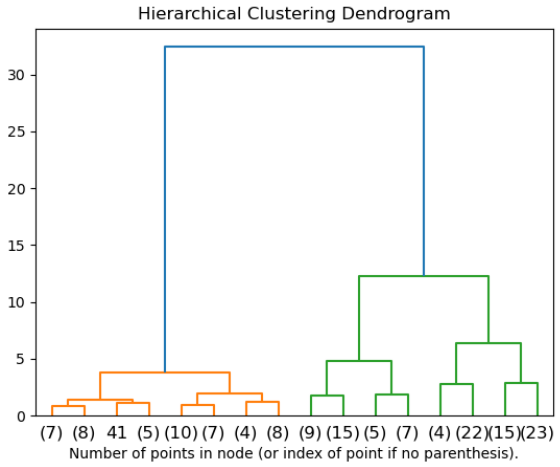


Image from: https://scikit-learn.org/stable/modules/clustering.html

Agglomerative clustering

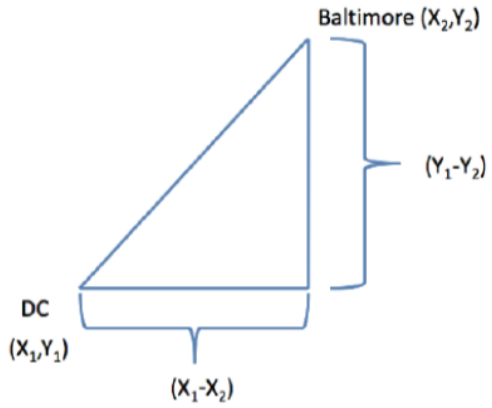
In practice the bottom-up approach works much better and thus we will only focus on agglomerative clustering

In agglomerative clustering, clusters are successively merged using a linkage criterion and based on a distance metric

`sklearn.cluster.AgglomerativeClustering`

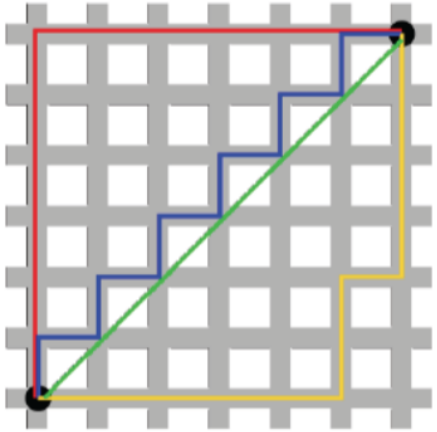
Distance metrics

Euclidean distance



$$\sqrt{(A_1 - A_2)^2 + (B_1 - B_2)^2 + \ldots + (Z_1 - Z_2)^2}$$

Manhattan (block) distance



$$|A_1 - A_2| + |B_1 - B_2| + \ldots + |Z_1 - Z_2|$$

Original slide by Evgenii Gilenko

Linkage criterion

Different linkage criterion result in di

On the right you can see the results c
clustering using different linkages

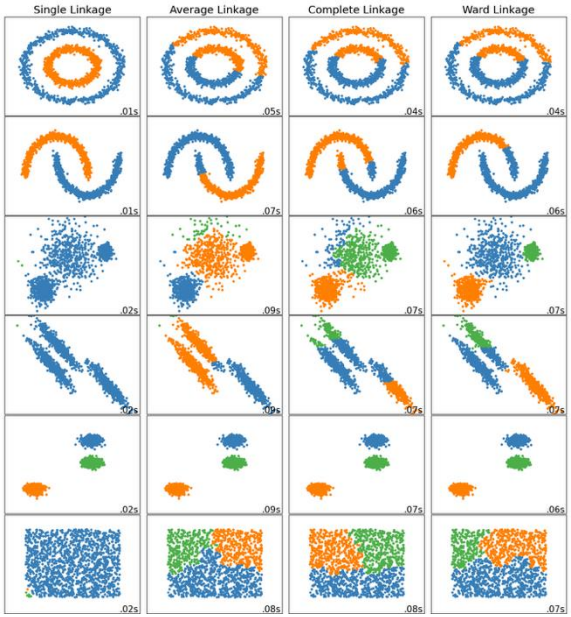


Image from: https://scikit-learn.org/stable/modules/clustering.html

Linkage criterion

Centroid: the distance between the centroids of each cluster

Single (“nearest neighbor”): the shortest distance between two points in each cluster

Complete (the ”furthest neighbor”): the longest distance between two points in each cluster

Average: the average distance between each two point in each two clusters

Ward: the sum of the squared distances from each point to the mean of the merged clusters

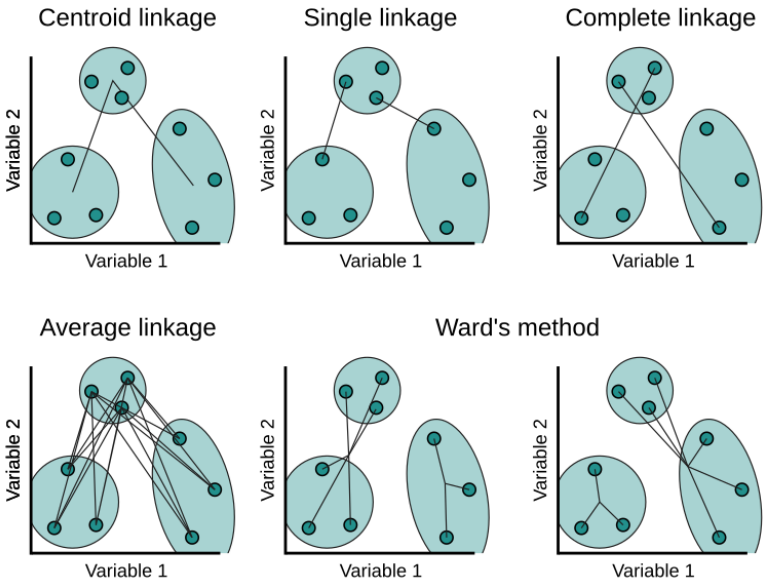


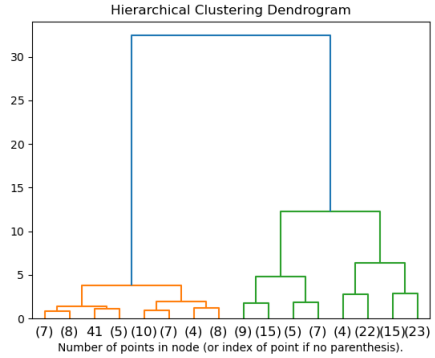
Image from this book

Dendrograms and scatterplots

The results of hierarchical clustering are often inspected with a dendrogram (top right)

The dendrogram is a tree diagram that shows the hierarchy and how the data it split into clusters

Scatter plots can also be used to plot the results of hierarchical clustering



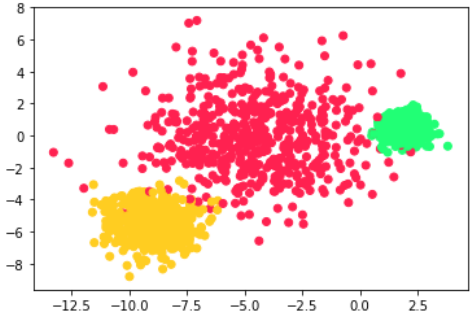


Image from: https://scikit-learn.org/stable/modules/clustering.html

Issues with hierarchical clustering

Infeasible with very large datasets

Influenced by order of datapoints

Sensitive to outliers

Documentation

K-Means:
https://scikit-learn.org/stable/modules/clustering.html#k-means

DBSCAN:
https://scikit-learn.org/stable/modules/clustering.html#dbscan

Agglomerative clustering
https://scikit-learn.org/stable/modules/clustering.html#hierarchical-clustering

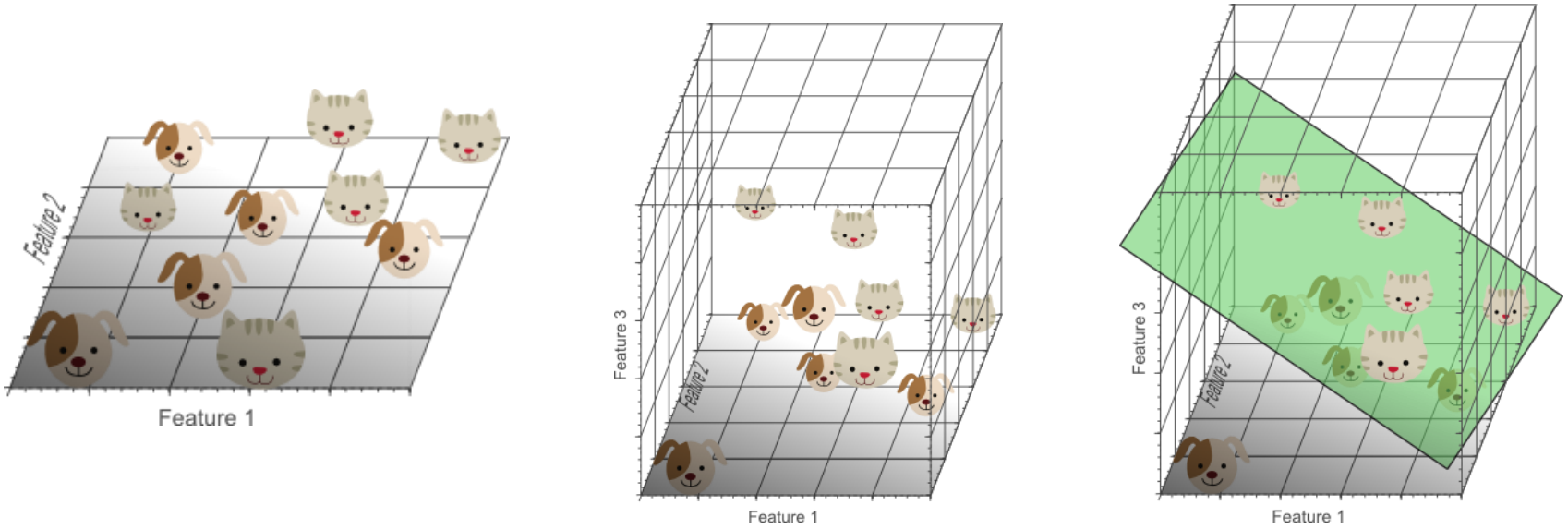
Dimensionality reduction

Dimensionality reduction

Dimensionality reduction is the transformation of data from a higher-dimensional space into a low-dimensional space, while maintaining as much of the original properties of data

High-dimensional data is mostly not optimal for ML. With dimensionality reduction, as an unsupervised preprocessing step the performance of supervised ML models can in some cases be improved

The curse of dimensionality



Images from https://www.visiondummy.com/2014/04/curse-dimensionality-affect-classification/

Principal component analysis (PCA)

PCA is used to determine the principal components that capture the most variance, resulting in a lower-dimensional projection of the data

As an example, if most of the variance in a 5-dimensional dataset can be represented in two dimensions, PCA let’s us extract these two principal components and continue working with them instead of the original higher-dimensional data

PCA

Going through *L7_2_pca.ipynb*

Conclusion

Conclusion

This lecture presented some common unsupervised machine learning methods and their applications

The focus was on clustering data and dimensionality reduction, although other applications of unsupervised ML also exist

Evaluation

Remember to complete the course evaluation and provide some feedback!

If 40% of the remaining participants in the course complete the evaluation, I will give 2 points of extra credit to every student in the course

Notes about the course

This was the first time the course was taught in its current form and the course will be organized again next year

Based on feedback and this experience, I plan on:

- Moving the lectures to the beginning of the week while keeping the exercises closer to the end of the week
- More strongly emphasize that the course is designed so that students study the materials (book, links) before each lecture
- Reducing the peer review component to 10% instead of 20% of the course grade

Why are the assignments so loosely worded?

In the mid-course feedback and in emails some students have hoped for more detailed instructions and that assignment would be broken down to more specific subtasks

There is a reason for the assignments being quite loosely worded, especially towards the end of the course

The purpose of this is to prepare you for work and for independently researching methods and tools. When working as an analyst, you will often be given tasks, and it is expected that you can solve them with limited support and supervision. Practicing dealing with uncertainty and ambiguity helps you prepare for that

As a sidenote, there seems to also be a generational difference, with younger generations preferring detailed instructions out of fear of losing points. I received many emails asking about the loss of less than 1% of the course total. I encourage you to focus on learning the topics rather than focusing on getting a perfect score, as only 90% is needed for the top grade

Questions, comments suggestions?